

[기술공통]

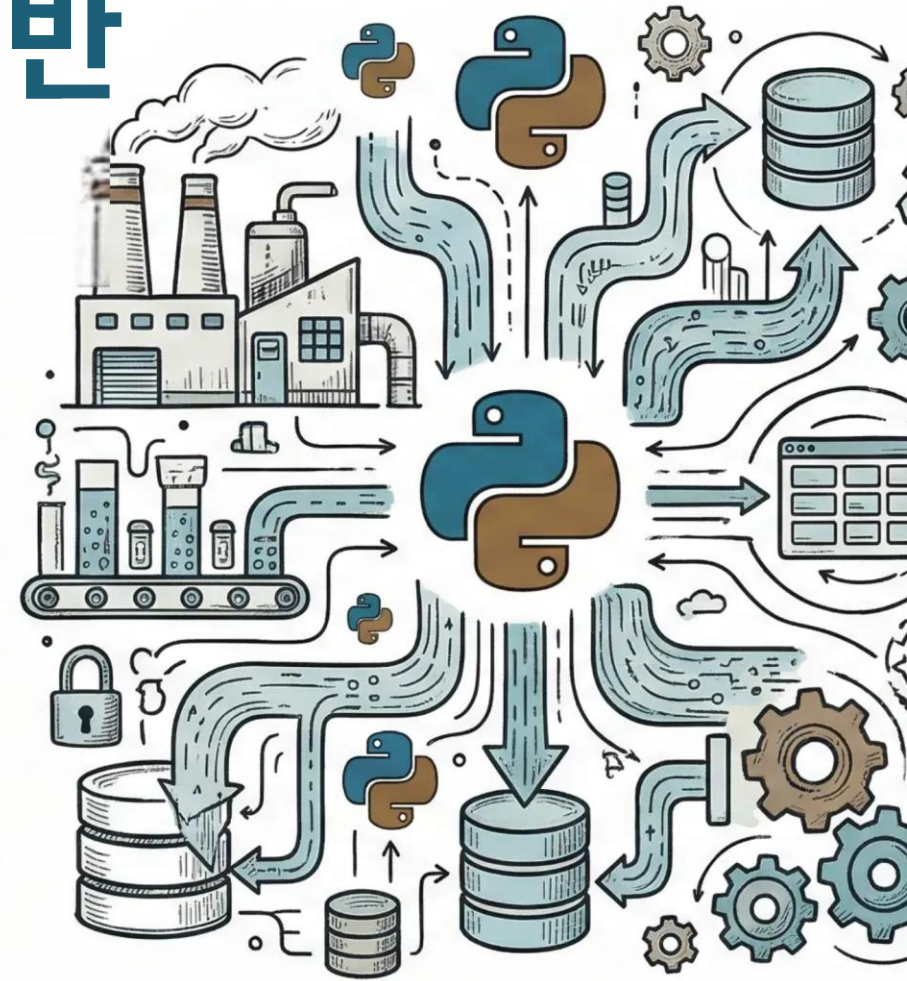
데이터 엔지니어링



제조 AX를 위한 Python 기반 데이터 엔지니어링 실무

**pandas · 데이터 전처리 · 데이터 수집 ·
SQLite 적재 · ETL · 품질 점검**

3주간의 실무 중심 과정으로 제조 현장의 데이터를
Python으로 처리하고 활용하는 방법을 학습합니다



지난주 결과를 다시 보기

•1주차 성과 확인

- AI4i 원본 데이터를 성공적으로 정제했습니다
- 데이터 구조와 품질을 파악했습니다
- 전처리 과정을 완료했습니다

•생성된 파일

- ai4i_cleaned.csv 파일이 준비되었습니다
- 이 파일이 2주차 실습의 출발점입니다

•오늘 할 일

- 정제된 데이터에 외부 정보를 추가합니다
- 여러 데이터 소스를 통합합니다

ai4i_cleaned.csv

"1주차에서 정제한 제조 데이터
오늘 이 파일을 기반으로
데이터 통합 작업을 시작합니다"



1주차 핵심 복습



- 데이터틀 읽고 구조를 이해했다

- AI4I 데이터셋의 행과 열 의미 파악
- DataFrame 기본 조작 방법 학습



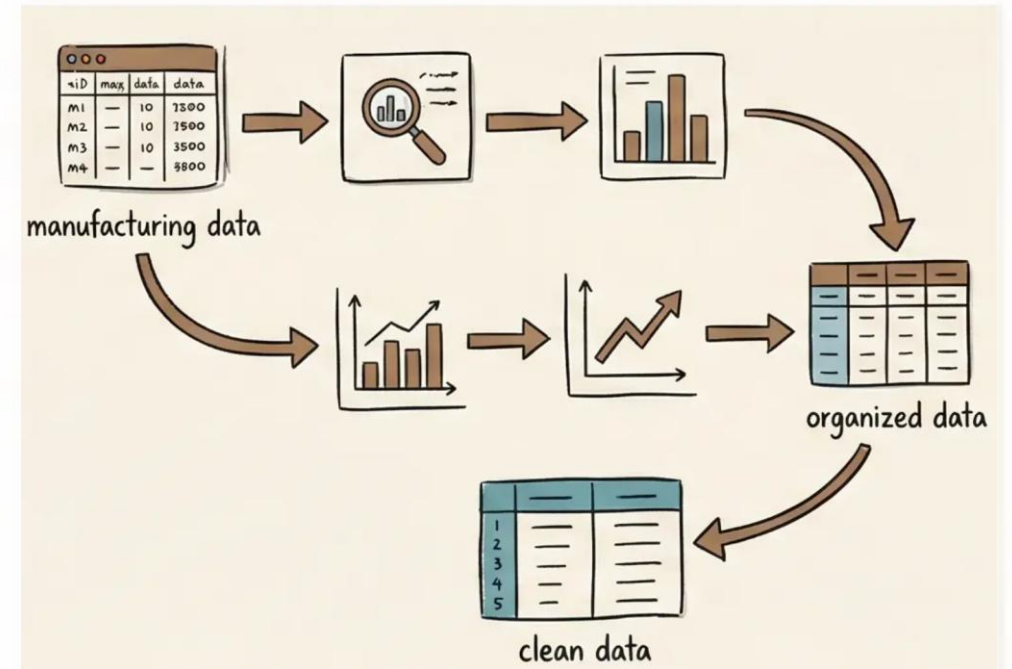
- EDA와 전처리를 수행했다

- 결측치, 중복, 분포 상태 점검
- 컬럼명 표준화 및 자료형 정리
- 파생 변수와 이상치 플래그 생성



- 정제 파일을 저장했다

- ai4i_cleaned.csv 파일 생성
- 다음 단계 작업을 위한 준비 완료



2주차 학습 목표

Primary Goal: 다양한 데이터 소스 통합 마스터하기

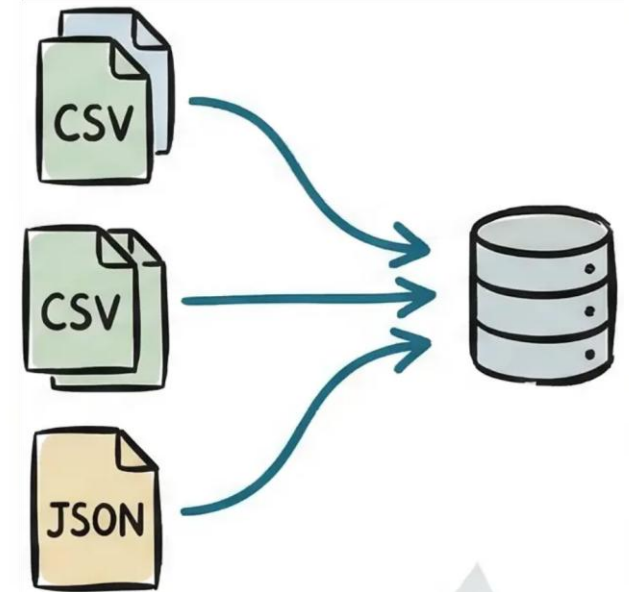
Key Learning Points

여러 CSV 파일에서 데이터 로드

기준정보와 메인 데이터 결합

JSON 형태의 외부 데이터 처리

통합된 데이터셋 생성 및 저장



이번 주 핵심 결과물

ai4i_enriched.csv - 모든 소스가 통합된
완성형 데이터셋

오늘 만들 결과물

다양한 데이터 소스를 통합하여 완성되는 ai4i_enriched.csv



1주차 정제 데이터

- ai4i_cleaned.csv
- 기본 제조 데이터
- 전처리 완료된 상태
- 메인 데이터셋 역할



기준정보 CSV

- 장비 유형 설명
- 코드 매핑 테이블
- 현장 해석 정보
- Type별 상세 정보



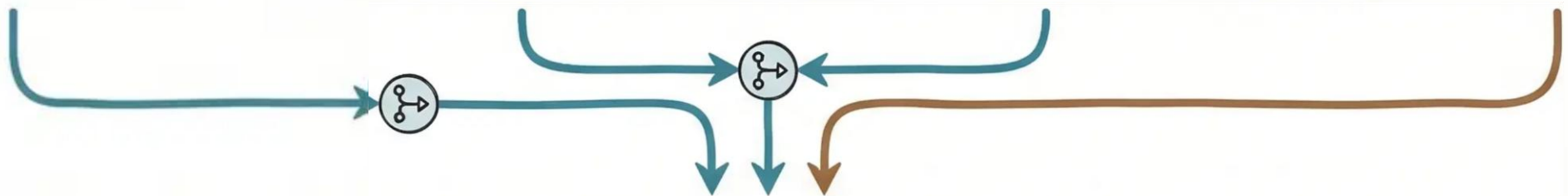
여러 CSV 파일

- 날짜별 로그 데이터
- 교대별 운전 정보
- 라인별 실적 데이터
- 추가 센서 정보



JSON 데이터

- API 응답 형태
- 외부 시스템 연동
- 실시간 상태 정보
- 보조 메타데이터



ai4i_enriched.csv



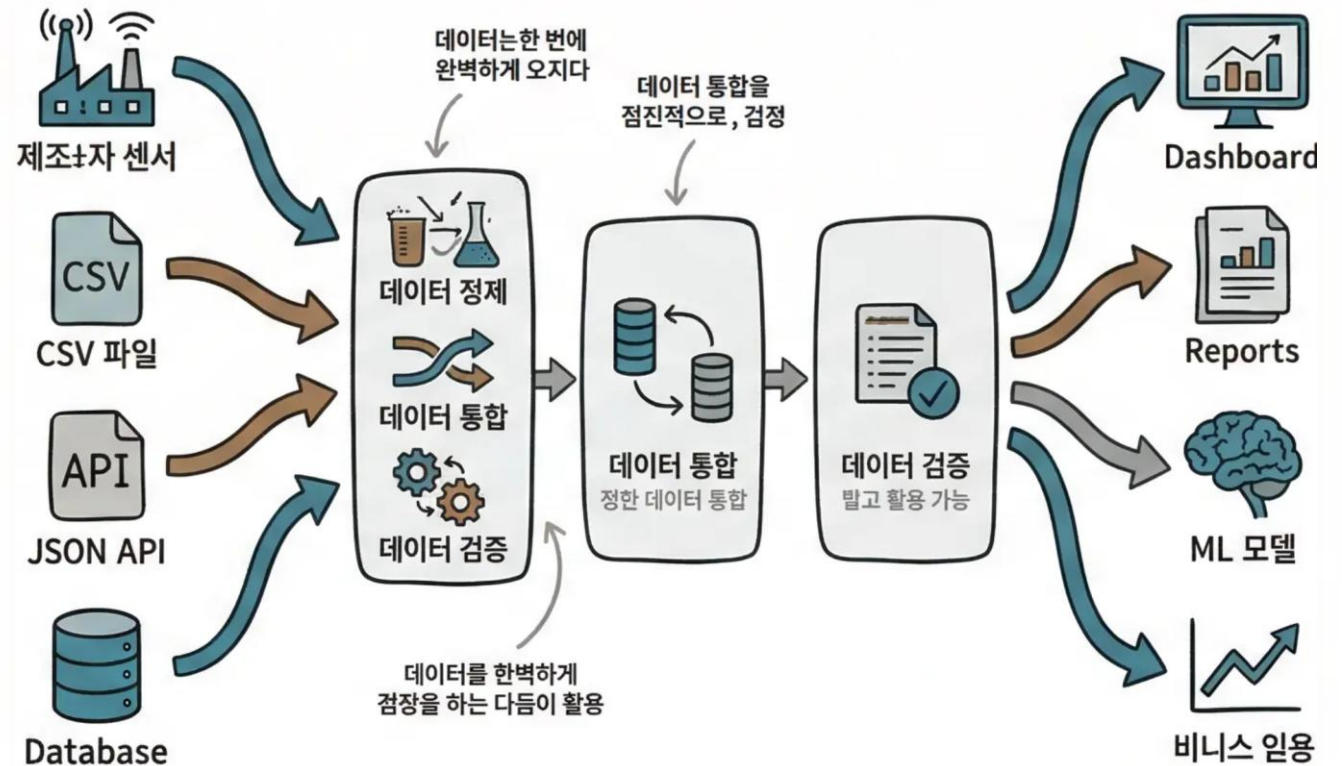
- 모든 소스가 통합된 완성본
- 분석 준비 완료 상태
- 3주차 SQLite 적재 대상

데이터 파이프라인의 무엇인가

데이터를 가져와 저장하고
처리하고 활용하는 자동화된
흐름

Key 특징

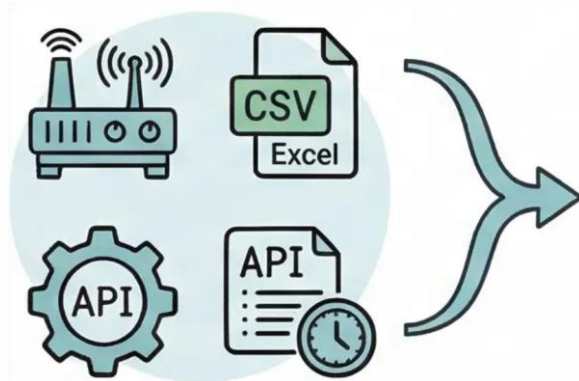
- 데이터는 한 번에 완벽하게 오지 않습니다
- 여러 단계를 거쳐 점진적으로 다듬어집니다
- 반복 가능하고 신뢰할 수 있는 프로세스입니다



수집, 저장, 처리, 활용의 흐름

데이터 파이프라인의 4단계 프로세스

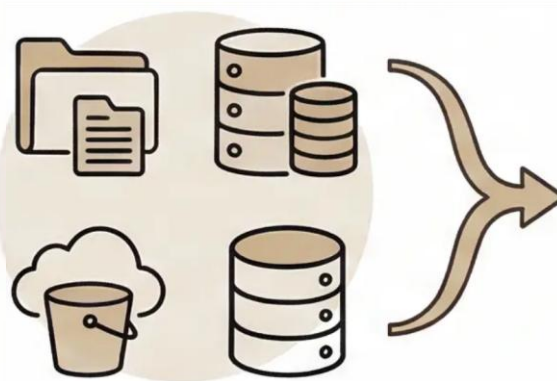
수집 (Collection)



수집 (Collection)

- 설비 센서 데이터
- CSV/Excel 파일
- API 응답 데이터
- 실시간 로그

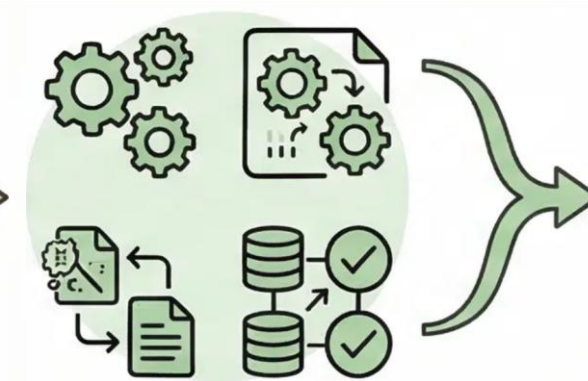
저장 (Storage)



저장 (Storage)

- 파일 시스템
- 데이터베이스
- 클라우드 저장소
- 데이터 레이크

처리 (Processing)



처리 (Processing)

- 데이터 정제
- 형식 변환
- 결합/통합
- 품질 검증

활용 (Utilization)



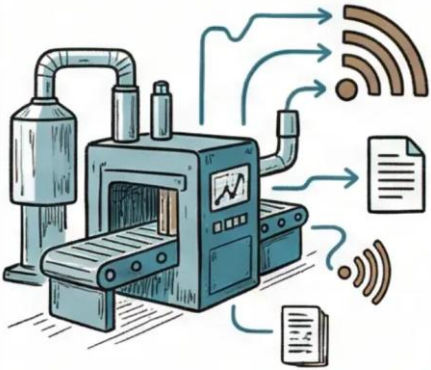
활용 (Utilization)

- 분석 리포트
- 대시보드
- AI/ML 모델
- 의사결정 지원

데이터는 한 번에 완성되지 않고, 단계별 변화를 통해 가치를 창출합니다

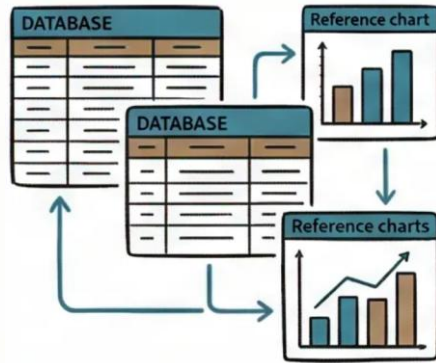
제조 현장의 데이터는 어디서 오는가

설비 로그 파일



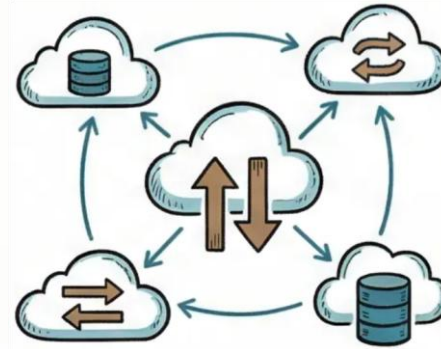
- 센서 데이터 실시간 기록
- 운전 상태 로그
- 알람 및 이벤트 기록

기준정보 테이블



- 제품 마스터 정보
- 설비 사양 데이터
- 작업 표준 정보

외부 JSON/API 데이터



- 외부 시스템 연동
- 실시간 API 통신
- 클라우드 서비스 데이터

실적 및 상태 데이터



- 생산 실적 정보
- 품질 측정 결과
- 현장 상태 보고

파일, API, 웹, DB의 차이

데이터 수집 방식에 따른 특징을 비교하고 각 입력원별 활용 시나리오를 이해합니다

파일 (Files)

특징 → 가장 흔한 입력원, 단순하고 직접적

장점 → 빠른 처리, 오프라인 가능

활용 → 일일 생산 로그, 품질 검사 결과

예시 → CSV, Excel, 텍스트 파일

웹 (Web)

특징 → 웹페이지에서 데이터 수집

장점 → 공개 정보 접근, 다양한 소스

활용 → 시장 정보, 공공 데이터

예시 → 웹 스크래핑, HTML 파싱

API (Application Programming Interface)

특징 → 시스템 간 실시간 데이터 통신

장점 → 자동화된 데이터 교환, 실시간 업데이트

활용 → 센서 데이터, 외부 시스템 연동

예시 → REST API, JSON 응답

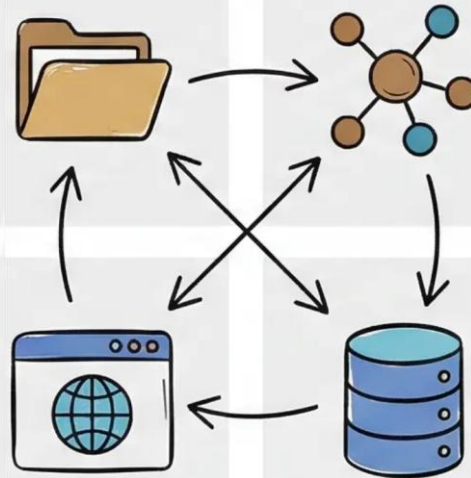
DB (Database)

특징 → 구조화된 대용량 데이터 저장소

장점 → 효율적 쿼리, 데이터 무결성

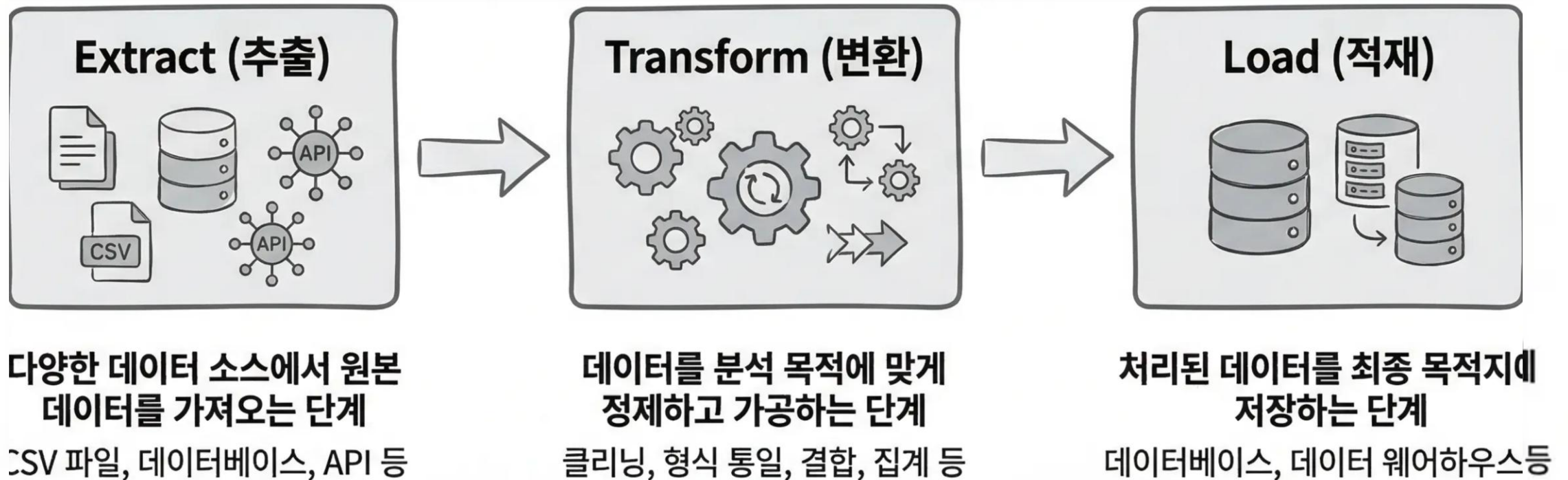
활용 → 기업 시스템, 이력 데이터

예시 → SQL 데이터베이스, NoSQL



ETL이란 무엇인가

데이터 엔지니어링의 핵심 프로세스



데이터 엔지니어링의 표준 워크플로우

ELT와 ETL의 차이

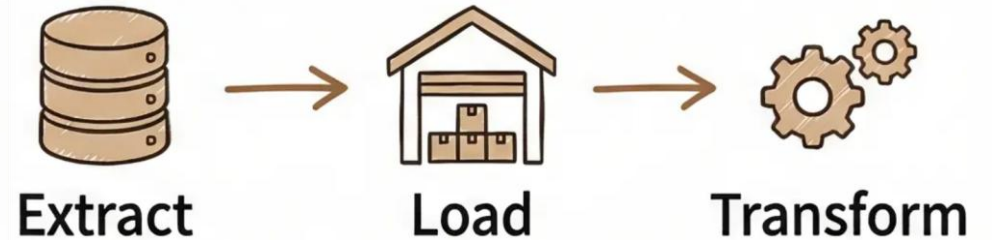
데이터 처리 순서에 따른 두 가지 접근 방식

ETL (Extract-Transform-Load)



- 추출 → 변환 → 적재 순서
- 데이터를 정리한 후 저장소에 넣기
- 변환 과정이 외부에서 진행
- 입문자가 이해하기 쉬운 구조
- 각 단계별 결과 확인 가능

ELT (Extract-Load-Transform)



- 추출 → 적재 → 변환 순서
- 원본 데이터를 먼저 저장소에 넣기
- 변환 과정이 저장소 내부에서 진행
- 대용량 처리에 유리
- 강력한 데이터베이스 성능 필요

이번 과정에서는 ETL 방식을 사용합니다

단계별 학습이 가능하고 각 과정의 결과를 명확히 확인할 수 있어 입문자에게 적합합니다

배치 처리와 스트리밍 처리

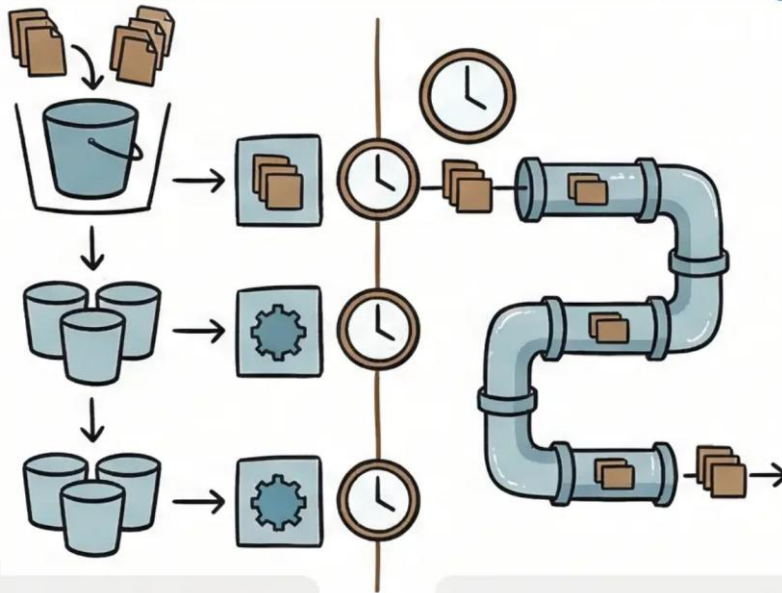
데이터 처리 방식의 두 가지 접근법



배치 처리 (Batch Processing)

- 데이터를 모아서 한 번에 처리
- 정해진 시간 간격으로 실행
- 대용량 데이터 처리에 적합
- 처리 시간이 길어도 괜찮은 경우

예 : 매일 밤 12시에 하루치 로그 파일 분석



스트리밍 처리 (Streaming Processing)

- 데이터가 들어오는 즉시 처리
- 실시간 또는 준실시간 처리
- 빠른 응답이 필요한 경우
- 연속적인 데이터 흐름 처리

예 : 센서 데이터 실시간 모니터링 및 알람

이번 과정에서는 입문자가 이해하기 쉬운 배치 처리 중심으로 실습합니다

이번 과정이 배치 중심인 이유



• 입문자 친화적 학습

- 전체 데이터 흐름을 단계별로 이해하기 쉽습니다
- 시작과 끝이 명확한 처리 과정을 따라갈 수 있습니다



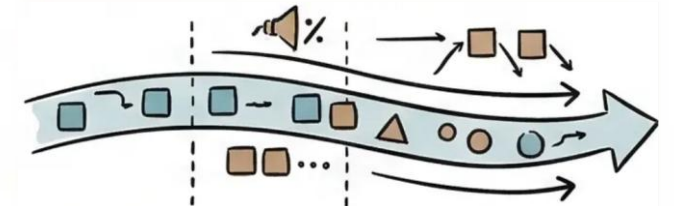
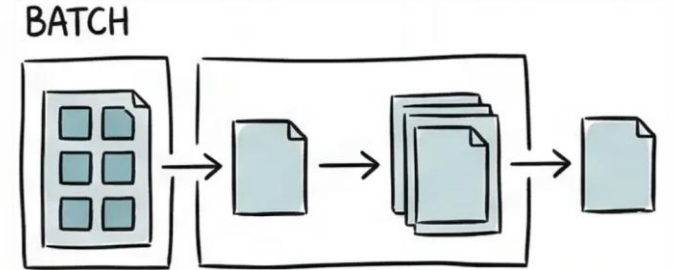
• 교육 환경의 안정성

- 네트워크나 실시간 연결 문제로 인한 실습 중단을 방지합니다
- 파일 기반 실습으로 예측 가능한 결과를 보장합니다



• 단계적 검증 가능

- 각 단계별로 결과를 확인하고 검증할 수 있습니다
- 문제 발생 시 원인을 쉽게 파악하고 해결할 수 있습니다



‘실무에서는 두 방식 모두 중요합니다’

배치 처리의 원리를 이해하면 스트리밍 처리도 더 쉽게 학습할 수 있습니다

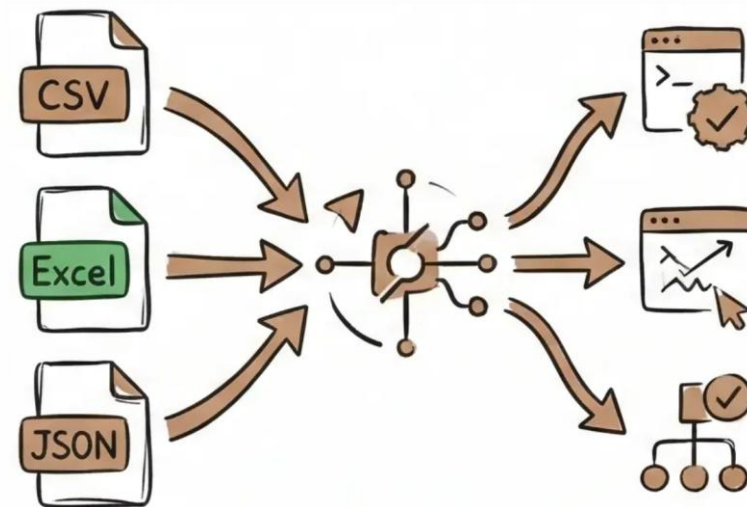
파일 기반 데이터 처리의 시작

• 현업에서의 파일 처리 현실

- CSV, Excel, JSON은 여전히 제조 현장의 주요 데이터 형태
- 시스템 간 데이터 교환의 가장 보편적인 방식

• 자동화의 출발점

- 반복적인 파일 처리 작업을 코드로 대체
- 수작업 오류 제거와 처리 시간 단축
- 확장 가능한 데이터 파이프라인의 기초



💡 실무 팁

파일 처리만 잘해도 일상 업무의 상당 부분을 자동화할 수 있습니다.
Python으로 시작하는 데이터 자동화의 첫걸음입니다.

CSV 읽기와 쓰기



• 가장 보편적인 데이터 교환 형식

- 단순하고 가벼운 구조
- 대부분의 시스템에서 지원
- 제조 현장에서 가장 많이 사용



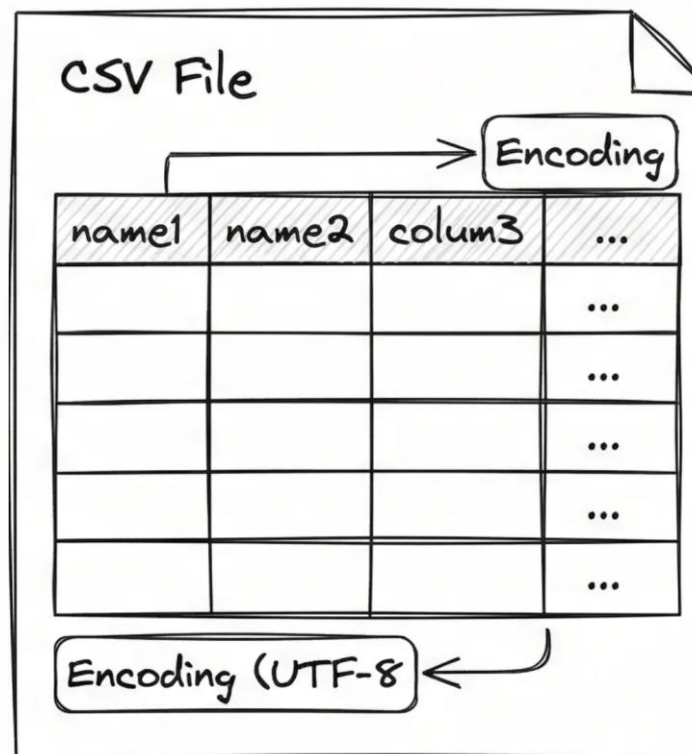
• 핵심 확인 사항

- 컬럼명의 정확성
- 인코딩 설정 (UTF-8, CP949)
- 구분자와 따옴표 처리



• 실무 팁

- 파일을 열어보기 전에 인코딩 확인
- 첫 번째 행이 헤더인지 데이터인지 점검
- 한글 깨짐 현상은 대부분 인코딩 문제

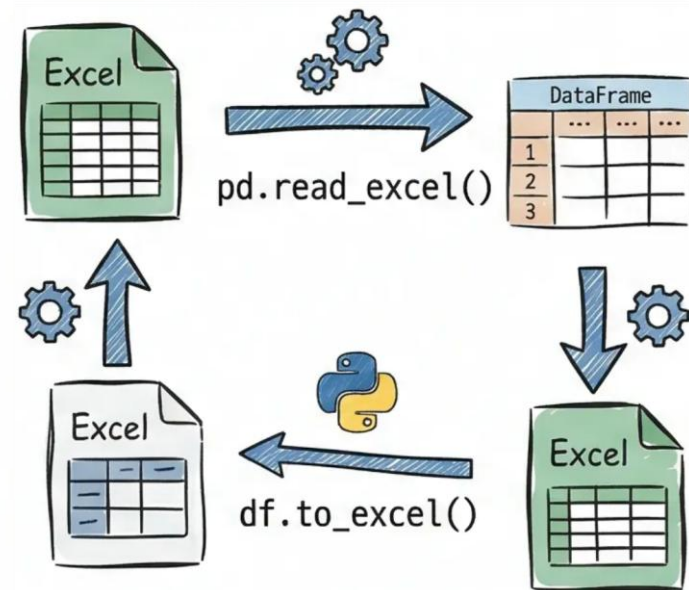


주의사항

CSV는 단순해 보이지만 인코딩과 구분자 문제로 인한 오류가 자주 발생합니다. 특히 한글이 포함된 제조 현장 데이터에서는 인코딩 확인이 필수입니다.

Excel 읽기와 쓰기

- **현업에서 Excel이 중요한 이유**
 - 사람이 보기 쉬운 형태
 - 현업 담당자들이 가장 익숙한 포맷
 - 보고서와 데이터 공유에 광범위하게 사용
- **Python으로 Excel 다루기**
 - `pd.read_excel()`: Excel 파일을 DataFrame으로 읽기
 - `df.to_excel()`: DataFrame을 Excel 파일로 저장
 - 시트 단위 처리 가능
- 자동화 관점에서의 고려사항
- CSV보다 처리 속도가 느림
- 메모리 사용량이 더 큼
- 대용량 데이터에는 부적합



실무 팁

- Excel은 사람이 확인하는 최종 결과물에 적합
- 자동화 처리 과정에서는 CSV 사용 권장
- 필요시에만 Excel로 변환하여 공유

Excel 읽기와 쓰기

- **현업에서 Excel이 중요한 이유**

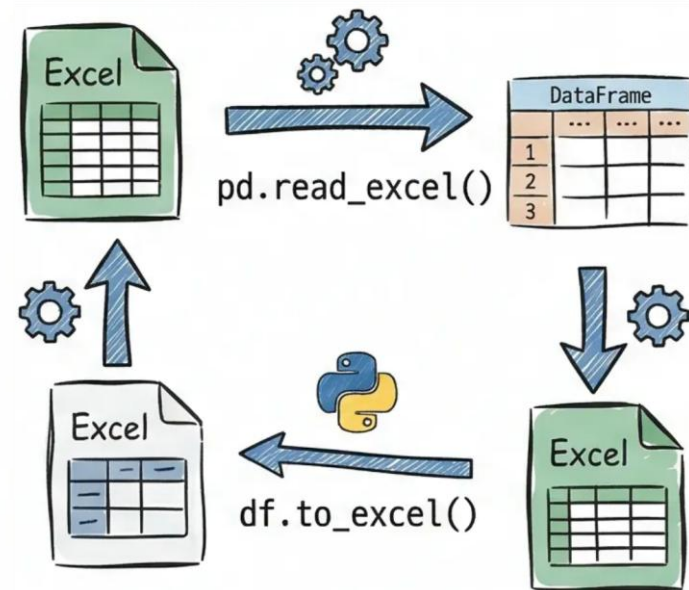
- 사람이 보기 쉬운 형태
- 현업 담당자들이 가장 익숙한 포맷
- 보고서와 데이터 공유에 광범위하게 사용

- **Python으로 Excel 다루기**

- `pd.read_excel()`: Excel 파일을 DataFrame으로 읽기
- `df.to_excel()`: DataFrame을 Excel 파일로 저장
- 시트 단위 처리 가능

- **자동화 관점에서의 고려사항**

- CSV보다 처리 속도가 느림
- 메모리 사용량이 더 큼
- 대용량 데이터에는 부적합



실무 팁

- Excel은 사람이 확인하는 최종 결과물에 적합
- 자동화 처리 과정에서는 CSV 사용 권장
- 필요시에만 Excel로 변환하여 공유

JSON 읽기와 쓰기

JSON의 특징

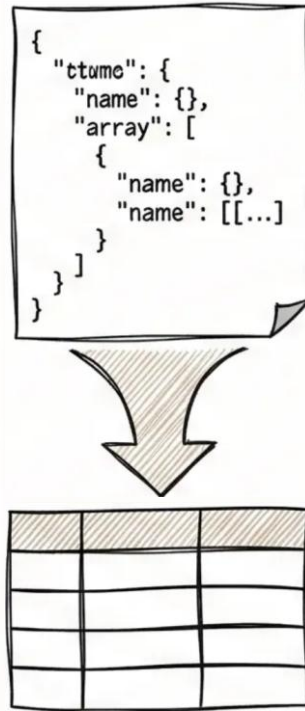


- API 응답에서 가장 많이 사용
- 구조화된 데이터 표현에 강함
- 중첩된 정보도 효율적으로 저장

JSON vs CSV 비교



- JSON: 계층적 구조 표현 가능
- CSV: 단순한 표 형태만 지원
- JSON: 웹 시스템과 호환성 우수

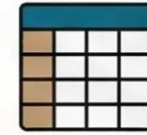


DataFrame 변환 과정



- `pd.read_json()` 함수 사용
- `json_normalize()` 로 중첩 구조 평면화
- 필대한 필드만 선택적 추출

실무 활용 팁



- 모든 데이터를 변환하지 말고 필요한 부분만
- 컬럼명 표준화 작업 병행
- 변환 후 데이터 타입 확인 필수

파일 포맷별 특징 비교

상황에 맞는 데이터 형식 선택하기



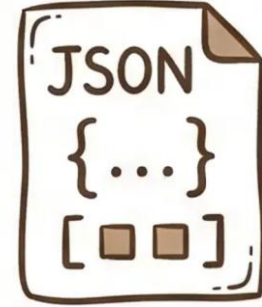
CSV 형식

- 장점: 단순하고 빠른 처리, 가벼운 파일 크기, 범용성 높음
- 단점: 구조적 표현에 한계, 데이터 타입 정보 없음
- 적합한 용도: 대용량 데이터 교환, 시스템 간 데이터 이동



Excel 형식

- 장점: 사람이 보기 편함, 차트와 서식 지원, 협업 친화적
- 단점: 자동화 시 처리 무거움, 버전 호환성 이슈
- 적합한 용도: 수동 보고서, 소규모 데이터 분석



JSON 형식

- 장점: 구조적 데이터 표현 강함, API 통신 표준, 중첩 구조 지원
- 단점: 사람이 읽기 어려움, 파일 크기 상대적으로 큼
- 적합한 용도: API 데이터 교환, 설정 파일, 웹 서비스

데이터의 규모, 처리 방식, 사용자 유형에 맞춰 가장 효율적인 포맷을 선택하여 사용하세요.

인코딩 문제를 이해하자

한글 깨짐의 주요 원인

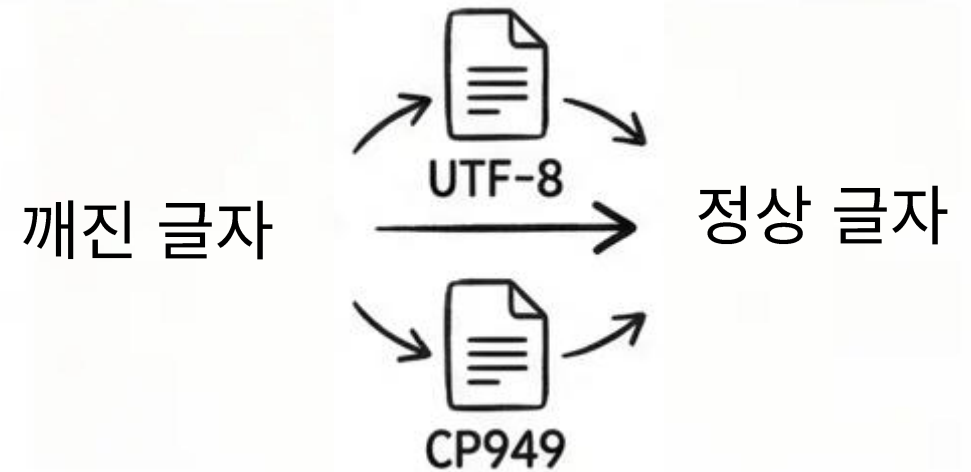
- 파일과 읽기 설정의 인코딩 불일치
- Windows 환경의 CP949와 UTF-8 충돌
- 저장 시 인코딩 설정 누락

문제 해결 방법

- 파일 저장 시: ``encoding='utf-8'`` 명시
- 파일 읽기 시: 인코딩 옵션 확인
- 오류 발생 시 다른 인코딩 시도

실무 팁

- 파일명에 한글 사용 자제
- 팀 내 인코딩 규칙 통일
- 저장 후 다시 읽어서 확인



주의사항

인코딩 문제는 데이터 유실로 이어질 수 있습니다.
항상 저장 후 재확인하는 습관을 기르세요.

경로와 파일명을 잘 관리하는 법

• 체계적인 폴더 구조가 중요한 이유

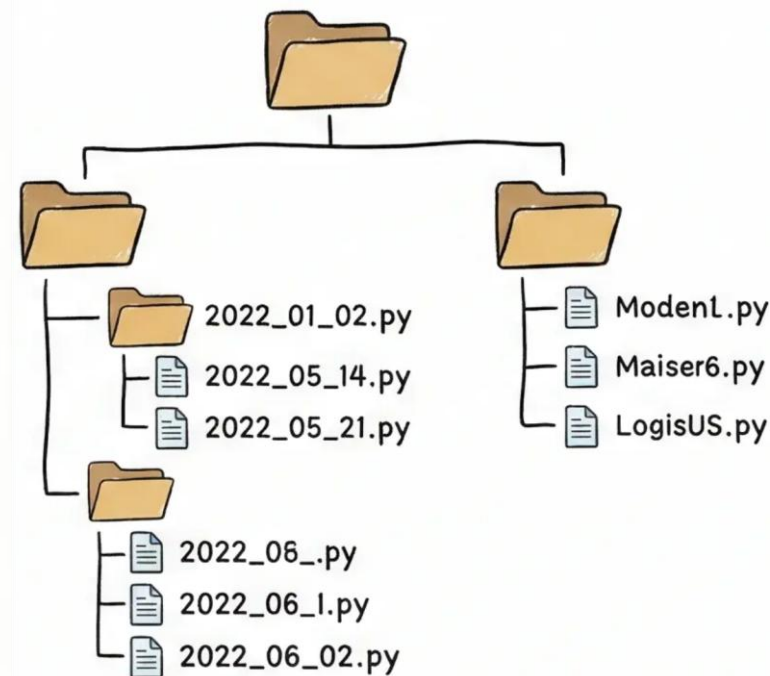
- 프로젝트가 커질수록 파일 찾기가 어려워집니다
- 일관된 구조는 팀 협업에 필수적입니다
- 자동화 스크립트 작성시 훨씬 쉬워집니다

• 파일명 규칙 설정하기

- 날짜는 YYYY-MM-DD 형식으로 통일
- 공백 대신 언더스코어(_) 사용
- 버전 관리를 위한 v1, v2 표기법

• 경로 설정의 실무 팁

- 절대 경로보다는 상대 경로 활용
- pathlib 모듈로 운영체제 독립적 경로 처리
- 하드코딩된 경로는 설정 파일로 분리



실무 경험담

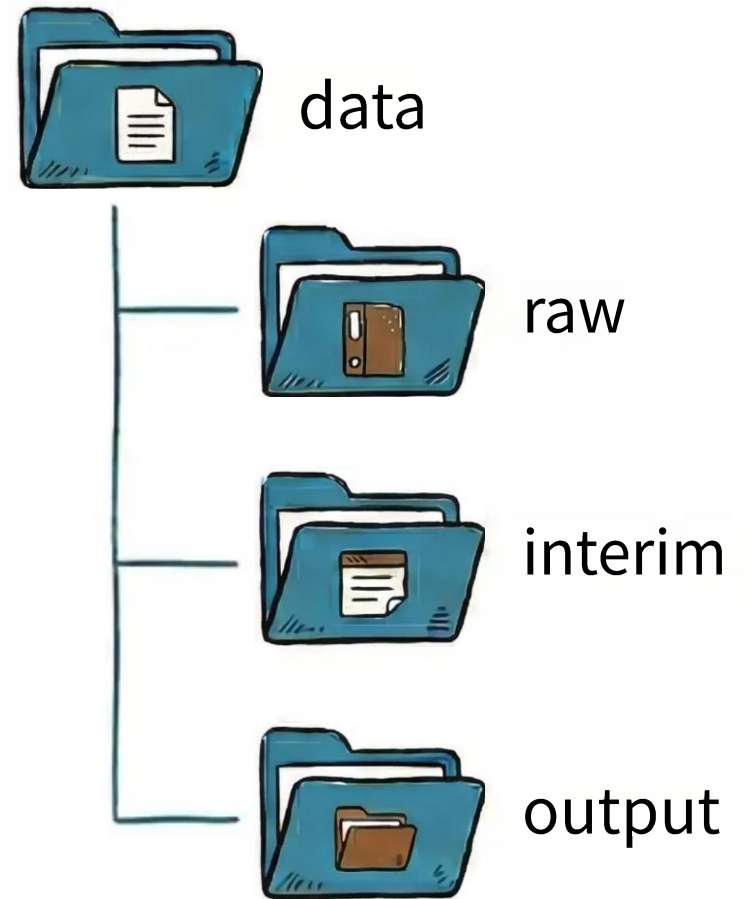
“파일명에 한글과 공백을 섞어 쓰다가 리눅스 서버에서 오류가 발생한 경험이 있습니다. 처음부터 영문과 언더스코어로 통일하는 것이 안전합니다.”

실무형 폴더 구조 예시

- data/raw/
- data/interim/
- data/output/

- raw 폴더 : 수정하지 않는 원본 파일들
- interim 폴더 : 처리 중간 단계 파일들
- output 폴더 : 완성된 결과 파일들

폴더를 나누면 실수로 원본을 지우는 일을
방지할 수 있고,
처리 단계별로 추적이 쉬워집니다.



pathlib로 파일 다루기

안전하고 직관적인 경로 관리

기존 문자열 방식의 문제점

- 운영체제별 경로 구분자 차이
- 문자열 연결 시 실수 발생 위험
- 경로 조작이 복잡하고 가독성 떨어짐
- 경로 존재 여부 확인의 번거로움

pathlib의 장점

- 객체 지향적 경로 처리
- 크로스 플랫폼 호환성
- 직관적인 메서드 제공
- 파일 시스템 작업 통합

```
from pathlib import Path
```

```
# 경로 생성
```

```
data_path = Path('data') / 'raw' / 'file.csv'
```

```
# 경로 정보 확인
```

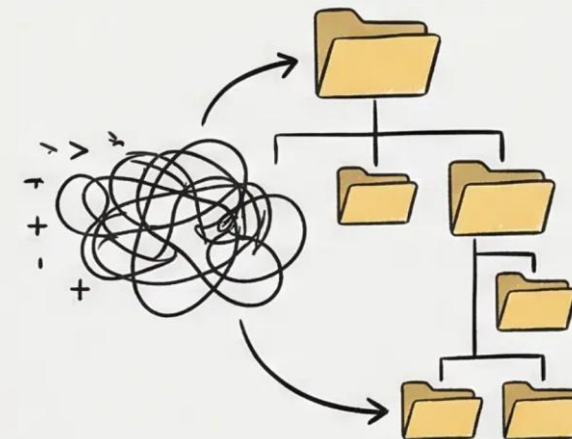
```
print(data_path.exists())
```

```
print(data_path.parent)
```

```
print(data_path.suffix)
```

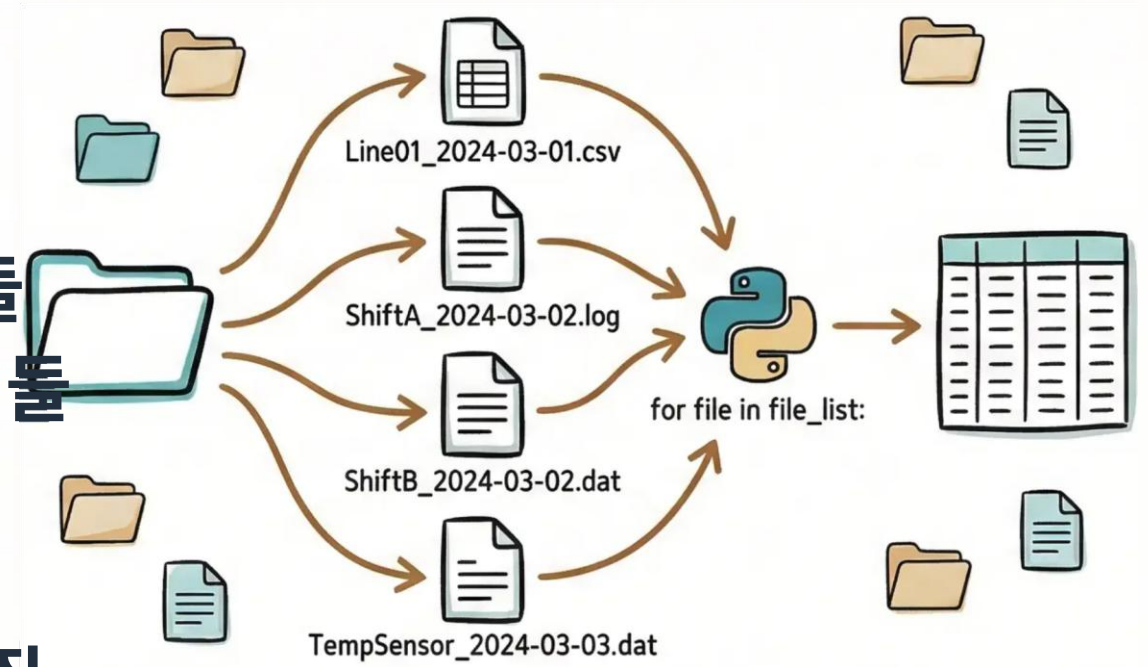
```
# 파일 목록
```

```
csv_files = Path('data').glob('*.csv')
```



폴더 안 여러 파일 읽기

- 제조 현장의 일반적인 상황
 - 날짜별로 분리된 생산 데이터 파일들
 - 라인별, 교대별로 나누어진 로그 파일들
 - 주기적으로 생성되는 센서 데이터 파일들
-
- 효율적인 처리 방법
 - 폴더 내 모든 파일 목록을 자동으로 수집
 - 반복문을 사용한 일괄 처리
 - 파일명 패턴을 활용한 선택적 읽기



실무 팁

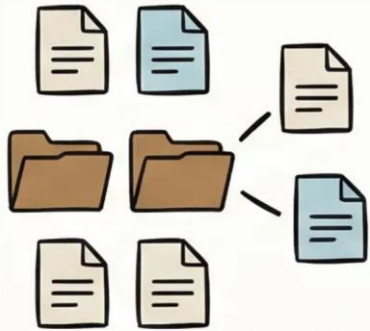
파일명에 날짜나 라인 정보가 포함되어 있다면, 이 정보를 메타데이터로 활용할 수 있습니다. 예: "Line01_2024-03-14.csv"

반복 처리의 기본 구조

파일 목록 수집부터 반복문을 통한 로드 및 최종 병합까지의 표준적인 코드 구조

1

파일 목록 수집

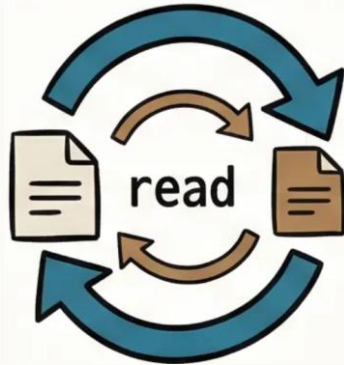


- 처리할 파일들을 먼저 찾아 리스트로 만들기
- glob 패턴이나 폴더 탐색 활용

```
files = glob.glob('data/*.csv')
```

2

반복문으로 하나씩 읽기

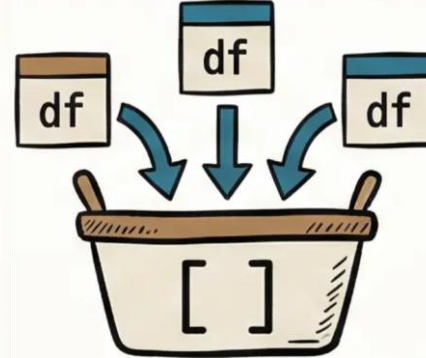


- for 문을 사용해 각 파일을 순차적으로 처리
- 읽은 DataFrame을 임시 리스트에 저장

```
for file in files:  
    df = pd.read_csv(file)  
    ...
```

3

리스트에 담기

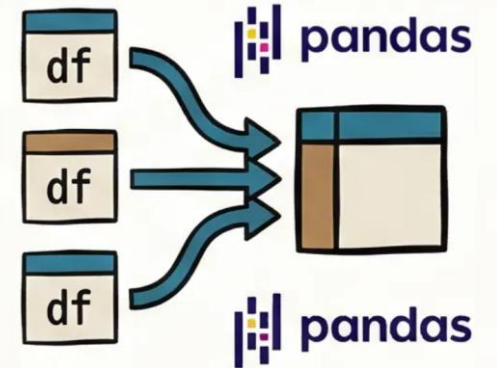


- 모든 DataFrame을 하나의 리스트로 수집
- 메모리 효율적인 중간 저장 방식

```
all_data = []  
...  
all_data.append(df)  
...
```

4

마지막에 합치기



- pandas.concat()으로 최종 통합
- 하나의 큰 DataFrame으로 완성

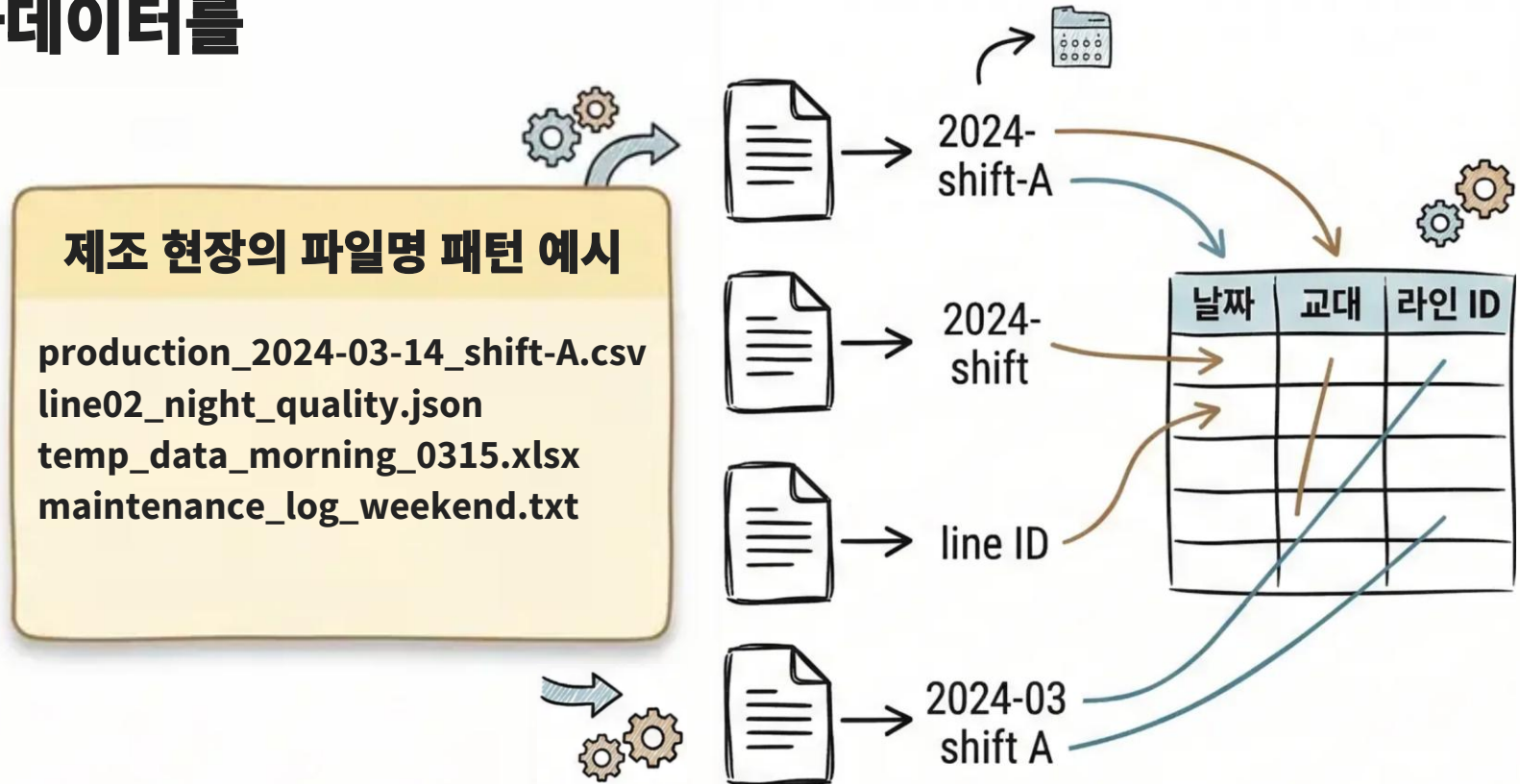
```
final_df = pd.concat(all_data)
```

Key Points

- 단계별로 자문자문 처리하는 것이 안전함
- 각 단계에서 검증하며 진행
- 실무에서 가장 많이 사용하는 패턴

파일명에서 정보 읽기

- 파일명 자체가 중요한 메타데이터를 담고있습니다
- 날짜, 교대, 라인 정보를 파일명에서 추출할 수 있습니다
- 추출된 정보를 데이터에 새로운 컬럼으로 추가합니다
- 실무 자동화에서 매우 자주 활용되는 기술입니다



여러 CSV를 하나로 합치기

• 구조 확인이 먼저

- 컬럼명과 자료형이 일치하는지 점검
- 키 컬럼의 일관성 확인

• 기본 합치기 흐름

- 파일 목록을 리스트로 수집
- 반복문으로 각 파일을 DataFrame으로 읽기
- pandas.concat()으로 새로 방향 결합

• 출처 추적하기

- 각 DataFrame에 파일명 컬럼 추가
- 어느 파일에서 온 데이터인지 기록
- 문제 발생시 원본 추적 가능

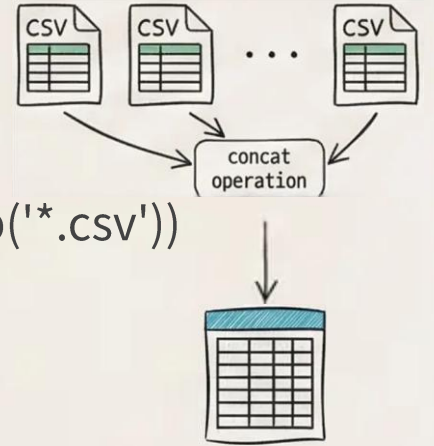
여러 CSV 파일 합치기 예시

```
import pandas as pd
from pathlib import Path
```

```
file_list = list(Path('daily_logs').glob('*.csv'))
df_list = []
```

```
for file_path in file_list:
    df = pd.read_csv(file_path)
    df['source_file'] = file_path.name # 출처 기록
    df_list.append(df)
```

```
combined_df = pd.concat(df_list, ignore_index=True)
print(f"합쳐진 데이터 크기: {combined_df.shape}")
```



💡 실무 팁

concat 후 반드시 전체 행 수를 확인하세요.

예상 행 수 = 각 파일 행 수의 합

병합 전 점검해야 할 것

✓ 컬럼명 일치 여부

- 같은 의미의 컬럼이 다른 이름을 사용하지 않는지 확인
- 대소문자, 공백, 특수문자 차이 점검



자료형 호환성

- 숫자 컬럼끼리 병합되는지 확인
- 문자열과 숫자가 섞이지 않았는지 점검



키(Key) 값의 일관성



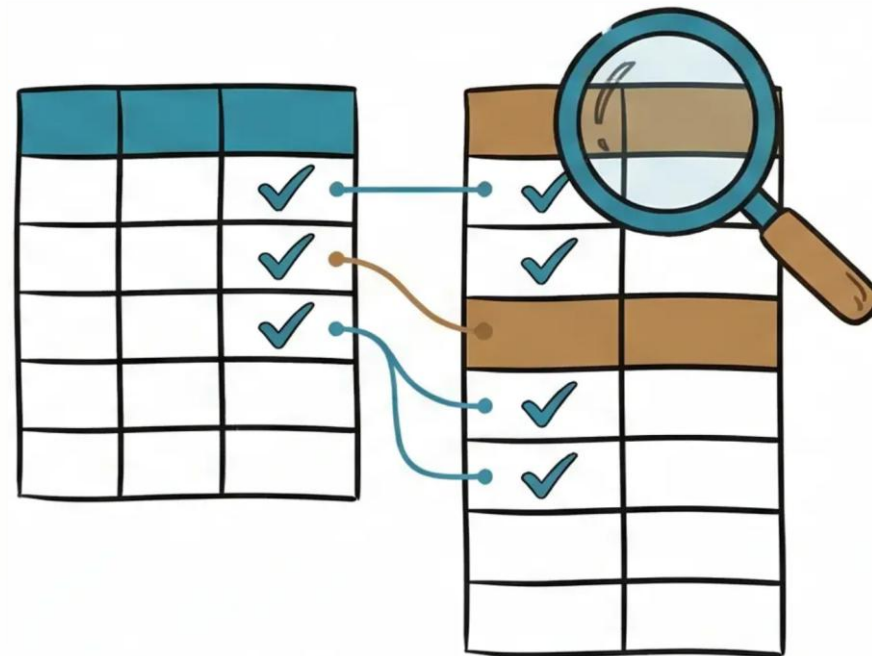
- 병합 기준이 되는 컬럼의 값 형식 통일
- 누락된 키 값이 있는지 사전 확인



행 수 예상치 계산



- 병합 후 예상되는 행 수를 미리 계산
- 중복이나 누락으로 인한 변화 예측

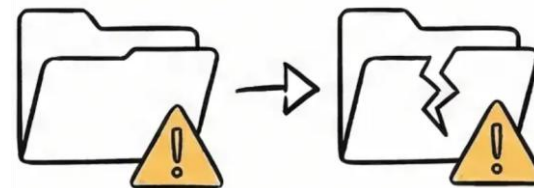


데이터 병합 실패의 80%는 사전 점검 부족에서 발생합니다.
병합 전 5분 투자가 나중에 1시간을 절약해줍니다.

파일 처리에서 자주 생기는 오류

• 경로 오류 (Path Errors)

- ⚠ 파일 경로를 잘못 지정했을 때, 상대 경로와 절대 경로 혼동
- ✅ 해결: pathlib 사용으로 안전한 경로 처리



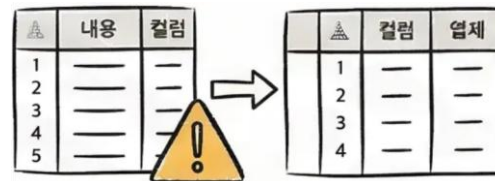
• 인코딩 문제 (Encoding Issues)

- ⚠ 한글 파일명이나 내용이 깨질 때, UTF-8과 CP949 인코딩 충돌
- ✅ 해결: encoding 매개변수 명시적 지정



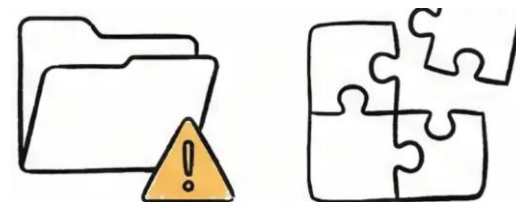
• 컬럼 불일치 (Column Mismatch)

- ⚠ 파일마다 컬럼명이나 순서가 다를 때, 예상 컬럼이 없거나 추가 컬럼 존재
- ✅ 해결: 병합 전 컬럼 구조 사전 확인



• 파일 누락 (Missing Files)

- ⚠ 예상한 파일이 폴더에 없을, 파일명 규칙 변경으로 인한 누락
- ✅ 해결: 파일 존재 여부 사전 점검



💡 실무 팁

오류가 발생하면 당황하지 말고 차근차근 확인하세요. 경로 → 인코딩 → 컬럼 순으로 점검하면 대부분 해결됩니다.

2주차 파일 실습 안내

•기존정보 CSV 결합

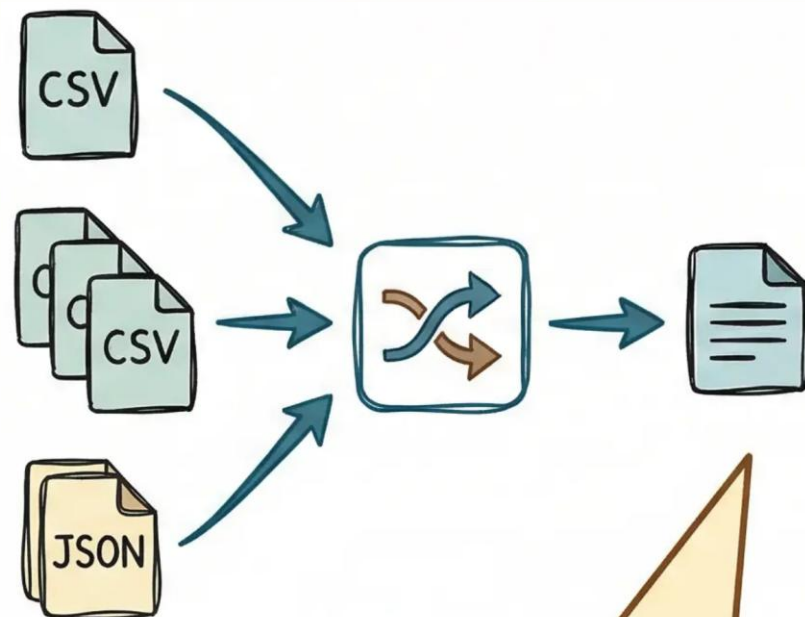
- ai4i_cleaned.csv에 타임별 설명 정보 추가
- merge() 함수로 left join 수행

•여러 파일 합치기

- 날짜별/라인별로 분리된 CSV 파일들 통합
- concat()으로 하나의 DataFrame 생성

•JSON 데이터 처리

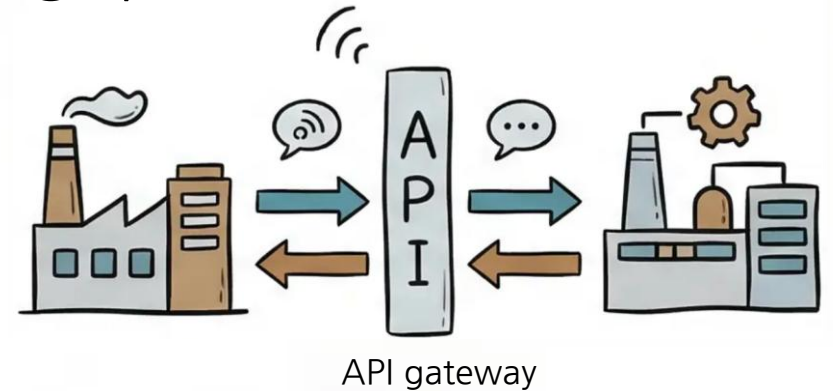
- 외부 시스템의 JSON 응답 데이터 읽기
- DataFrame 형태로 변환 후 결합



최종 목표
모든 소스를 통합한
ai4i_enriched.csv 파일 생성

API란 무엇인가

- API는 Application Programming Interface의 줄임말
- 프로그램과 프로그램이 서로 소통하는 통로
- 사람이 아닌 시스템끼리 자동으로 데이터를 주고받는 방식
- 제조 현장에서는 여러 시스템이 연결되어야 함
- 설비 모니터링 시스템 ↔ 생산 관리 시스템
- 품질 검사 시스템 ↔ 데이터 분석 시스템
- 실시간 데이터 교환으로 효율성 향상

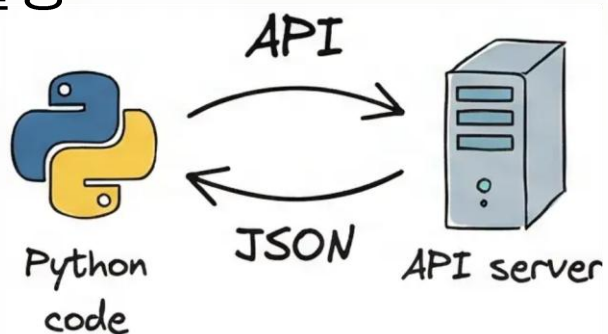


“API는 제조 현장의 다양한 시스템을 연결하는
디지털 다리 역할을 합니다.”

requests로 API 호출하기

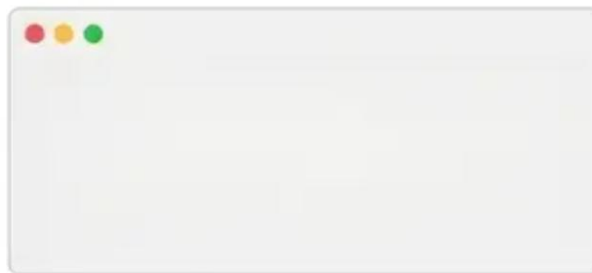
API 호출의 기본 개념

- Python requests 라이브러리 활용
- HTTP GET/POST 요청 방식
- 응답 데이터 JSON 형태로 수신



실제 API 호출 예시

- `requests.get(url)` 기본 문법
- 헤더와 파라미터 설정
- 응답 상태 코드 확인



이번 강의에서의 접근 방식

- 실제 API 대신 JSON 샘플 파일 사용
- 네트워크 환경 변수 최소화
- 안정적인 시스템 환경 구축

JSON 샘플 활용의 장점

- 일관된 데이터로 반복 학습 가능
- 외부 의존성 없는 안정적 실습
- 핵심 개념 이해에 집중

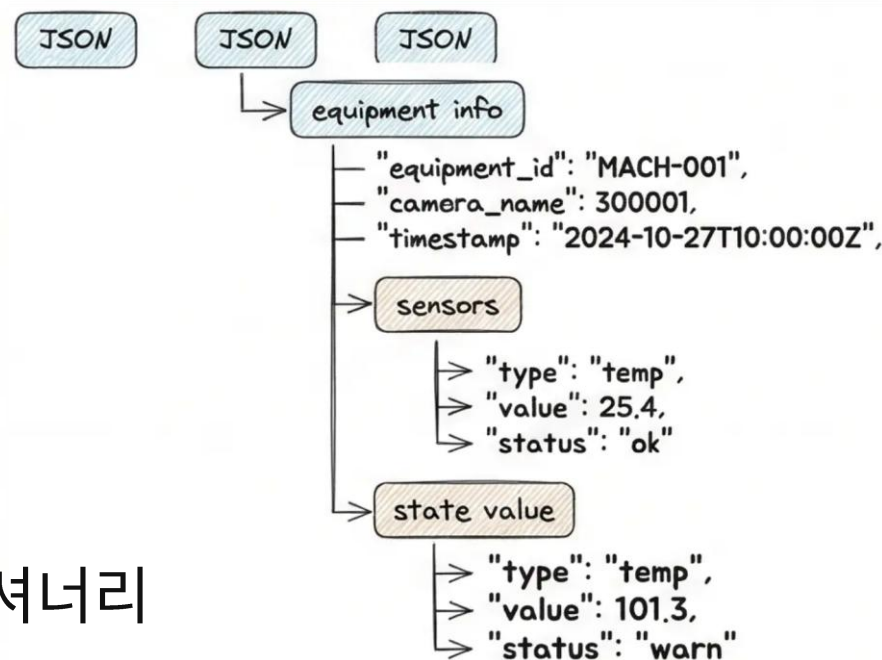
JSON 응답 구조 이해하기

JSON의 기본 구성 요소

- 딕셔너리 {}: 키-값 쌍으로 데이터 저장
- 리스트 []: 순서가 있는 데이터 집합
- 중첩 구조: 딕셔너리 안에 리스트, 리스트 안에 딕셔너리

제조 데이터 JSON 예시

- 장비 정보, 센서 데이터, 상태 값들이 계층적으로 구성
- API 응답에서 필요한 필드만 선택적 추출 가능



```
{
  "equipment_id": "MACH-001",
  "timestamp": "2024-10-27T10:00:00Z",
  "sensors": [
    { "type": "temp", "value": 25.4, "status": "ok" },
    { "type": "pressure", "value": 101.3, "status": "warn" }
  ]
}
```

“JSON은 복잡해 보이지만 결국 표(DataFrame)로 변환하는 것이 핵심입니다”

응답 데이터를 DataFrame으로 바꾸기

JSON을 표 형태로 변환하여 분석 효율성 향상

JSON 구조 이해하기

- 딕셔너리와 리스트의 중첩 구조
- 필요한 데이터 필드 식별

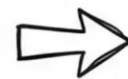
변환 핵심 방법

- `pd.json_normalize()` 함수 활용
- 중첩된 데이터 평면화 처리
- 컬럼명 자동 생성 규칙

실무 변환 팁

- 불필요한 필드 제외하기
- 일관된 데이터 타입 확보
- 결측값 처리 방안

```
{  
  "results": [  
    {  
      "id": 1,  
      "details": {  
        "name": "A"  
      }  
    },  
    {  
      "id": 2,  
      "details": {  
        "name": "B"  
      }  
    }  
  ]  
}
```



	id	details_name
1	1	A
2	2	B



핵심 포인트

JSON → DataFrame 변환 후에는 CSV와 동일한 방식으로 merge, 필터링, 집계가 가능합니다.

웹 데이터 수집의 기본 개념

- 웹도 결국 필요한 정보를 읽어 표로 정리하는 과정
 - HTML에서 원하는 데이터 추출
 - 비구조화된 정보를 구조화된 형태로 변환
- 데이터 엔지니어링 관점에서의 웹 수집
 - 다양한 소스 중 하나의 입력원
 - 최종 목표는 분석 가능한 DataFrame 생성
- 핵심은 데이터 구조화
 - 웹 페이지 → 필요한 정보 식별 → 표 형태로 정리
 - CSV, JSON과 같은 최종 처리 흐름



이번 과정에서는 개념만 연결
웹 스크래핑의 세부 기술보다는 전체 데이터 파이프라인에서의 역할 이해에 집중

표 형태 데이터를 읽는 흐름

핵심 개념:

다양한 형식의 데이터를 하나의 공통 구조로 통합

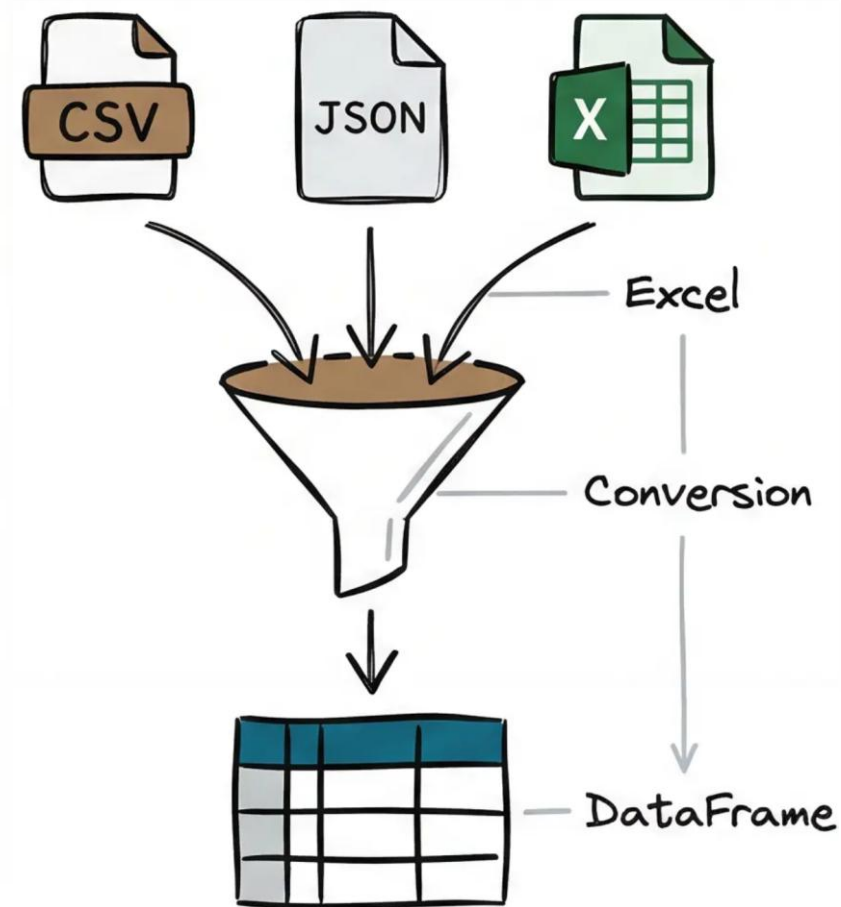
공통 목표:

모든 데이터를 DataFrame으로 변환

처리 흐름:

읽기 → 변환 → 통합 → 분석

- CSV도 표, JSON도 표로 바꾸면 같은 흐름
- DataFrame이 공통 처리 도구가 된다
- 형식이 달라도 최종 목표는 동일하다
- 통합 후에는 동일한 방식으로 처리 가능



핵심 개념 : 다양한 형식의 데이터를 하나의 공통 구조로 통합

외부 데이터와 기존 데이터 결합하기

제조 현장의 데이터는 한 곳에만 있지 않습니다.

메인 데이터에 다양한 외부 소스를 결합하여 더 풍부한 분석이 가능해집니다.



메인 제조 데이터: AI4I 센서 및 운전 변수 (1주차 정제 결과)



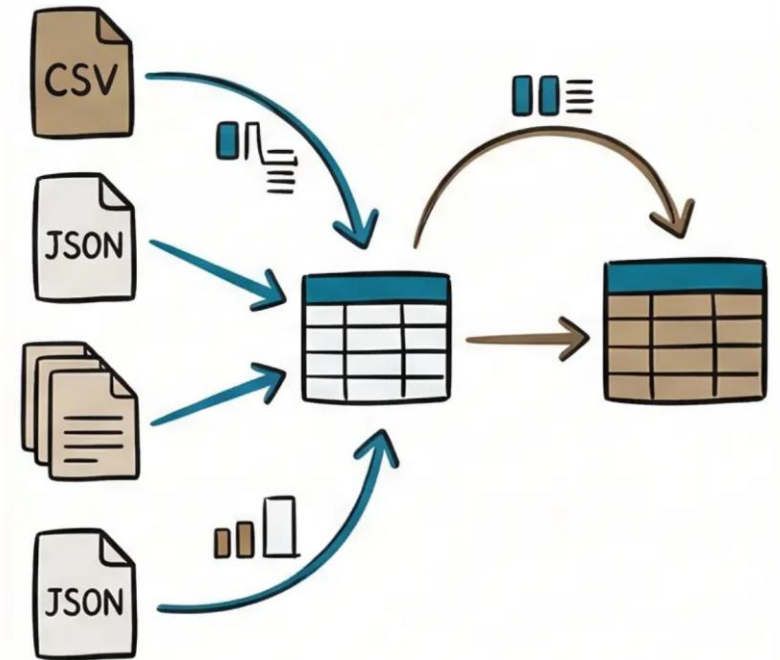
기준정보 CSV: 장비 유형별 상세 설명과 속성



여러 파일 로그: 교대별, 라인별 추가 운전 데이터



JSON 보조 정보: 외부 시스템의 실시간 상태 데이터



데이터 결합의 핵심 전략

공통 키를 중심으로 단계적 결합 → 결합 후 품질 검증 →
통합 데이터로 새로운 인사이트 발견

merge로 데이터 결합하기

- pandas의 merge() 함수는 두 개의 DataFrame을 공통 키(열)를 기준으로 결합할 수 있습니다.
- 이는 SQL의 JOIN 연산과 같은 개념이며, 주로 Left Join 방식을 사용하여 기준이 되는 메인 데이터에 다른 데이터의 정보를 덧붙일 때 활용합니다.
- **공통 키 확인:** 두 DataFrame 간에 공통으로 존재하는 키(열)를 식별합니다.
- Left Join 방식: 왼쪽(DataFrame 1)의 모든 행을 유지하면서, 오른쪽(DataFrame 2)의 해당 키 값이 일치하는 데이터를 결합합니다.
- **결합 후 검증:** 결합된 결과의 행 수나 누락된 값(null 값) 등을 확인해 데이터가 올바르게 병합되었는지 검증합니다.

실습 예시

```
df_merged = pd.merge(  
    df_main,          # 메인 데이터  
    df_reference,     # 기준정보  
    on='Type',        # 공통 키  
    how='left'        # Left Join  
)
```

Type	Data
A1	M350
M2	20-84
M3	30-84
M4	M340

Type	Description
CH1	제조정보 아타
CH2	기준정보 데안

	Type	Description
1	A1	제조정보 비2정보
2	M2	기준정보
3	M3	기준정보
4	CH2	제조정보 아타 기준정보 데안

결합 전후 row 수 비교하기

데이터 무결성 검증의 핵심 단계

• 원본 데이터 행 수 기록

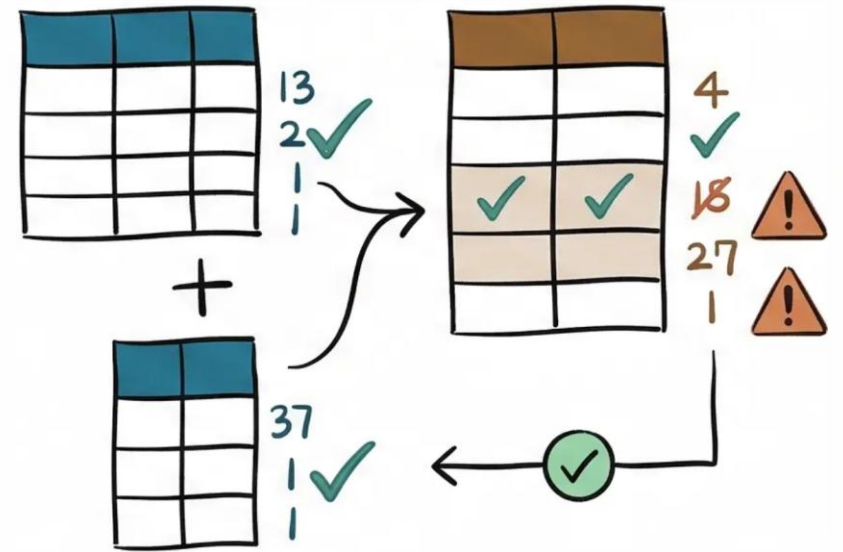
- merge 전 각 DataFrame의 행 수를 먼저 확인
- 예상되는 결과 행 수를 미리 계산

• 결합 후 행 수 검증

- left join의 경우 왼쪽 DataFrame 행 수와 동일해야 함
- inner join의 경우 양쪽 모두에 있는 데이터만 남음

• 결측치 변화 모니터링

- merge 후 새로 생긴 결측치 확인
- 예상치 못한 NaN 값 증가 점검



⚠ 주의사항

- 행 수가 예상과 다르면 키 매칭 문제 의심
- 결측치 급증은 데이터 품질 이슈 신호
- 검증 없는 merge는 분석 결과를 왜곡시킴

2주차 종합 실습 정리

데이터 통합의 전체 흐름을 완성했습니다

실습 단계별 흐름

1단계: 1주차 결과물 불러오기

- ai4i_cleaned.csv 파일 로드
- 데이터 상태 확인

2단계: 기준정보 붙이기

- 마스터 데이터와 left join
- Type 컬럼 기준 결합

3단계: 여러 파일 합치기

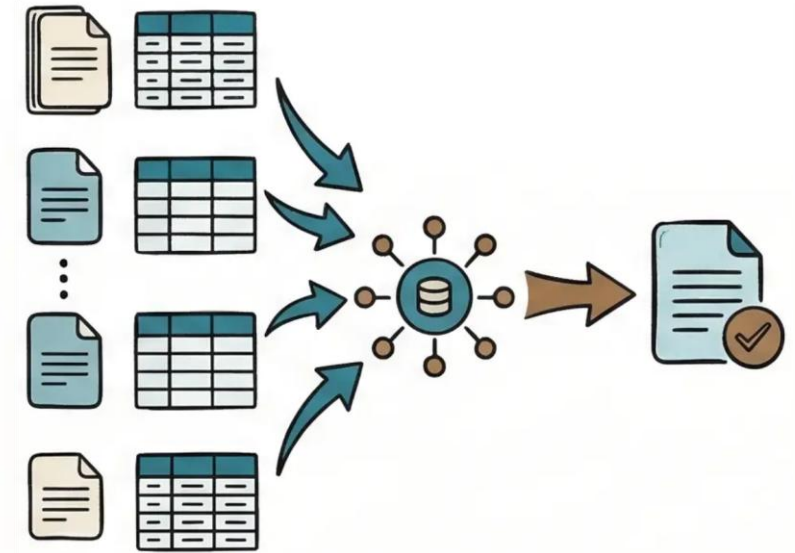
- 폴더 내 CSV 파일들 일괄 처리
- concat으로 통합

4단계: JSON 데이터 읽기

- API 응답 형태 데이터 처리
- DataFrame으로 변환

5단계: 통합 CSV 저장

- ai4i_enriched.csv 생성
- 다음 주차 준비 완료



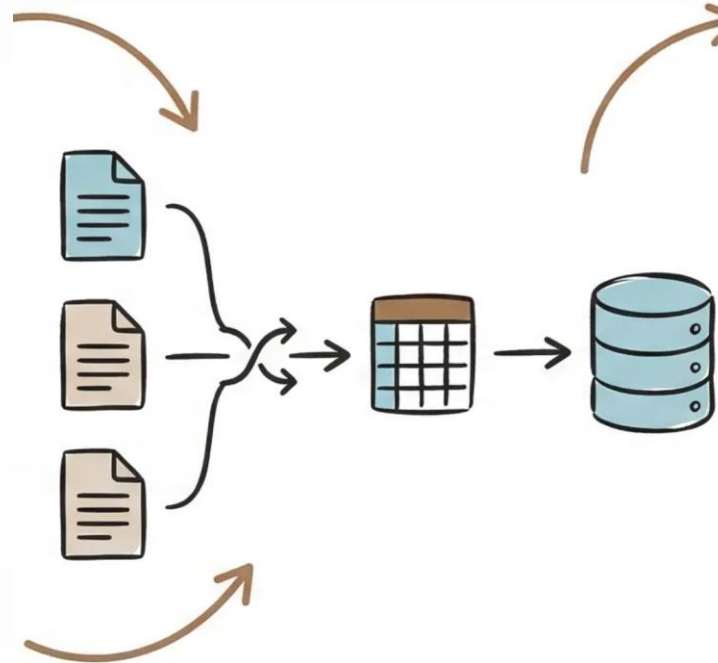
핵심 포인트

- ✓ 여러 데이터 소스를 하나로 통합
- ✓ 파일 기반 ETL 흐름 경험
- ✓ 데이터 결합 전후 검증 습관
- ✓ 최종 결과물 저장 및 확인

2주차 요약과 다음 주 예고

이번 주 달성한 것들

- 여러 데이터 소스를 성공적으로 읽었습니다
- 기준정보와 외부 데이터를 결합했습니다
- JSON 데이터를 DataFrame으로 변환했습니다
- 통합 파일 `ai4i\_enriched.csv`를 생성했습니다



다음 주에 배울 내용

- SQLite 데이터베이스 연결과 생성
- 통합 데이터의 테이블 적재
- SQL 쿼리를 통한 데이터 검증
- 품질 점검과 로그 기록 자동화

데이터는 한 파일에만 있지 않습니다. 여러 소스를 표 형태로 바꿔 결합할 수 있고, 이제 데이터베이스에 안전하게 저장할 준비가 되었습니다.